



lab3 标准单元版图自动生成的模拟退火实现

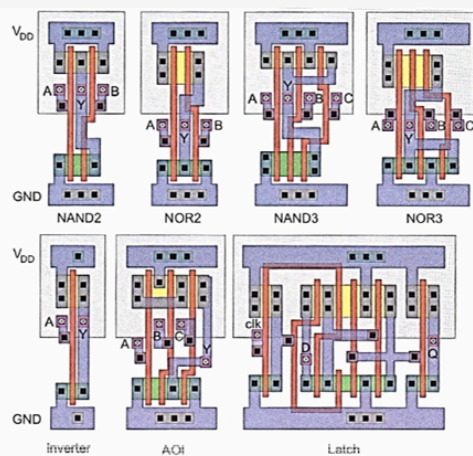
1. 实验目的

- 了解标准单元布局的相关知识。
- 理解模拟退火算法的基本框架与思想,并用模拟退火解决标准单元布局的问题。
- 掌握局部搜索中“邻域产生-合法性检查-目标函数-接受准则-降温策略”的完整流程。
- 本次实验**不要求同学自己设计布局算法**,布局算法已经在指南和代码中给出,同学只需**学习和使用**提供好的辅助函数完成**模拟退火主框架**代码的补全即可

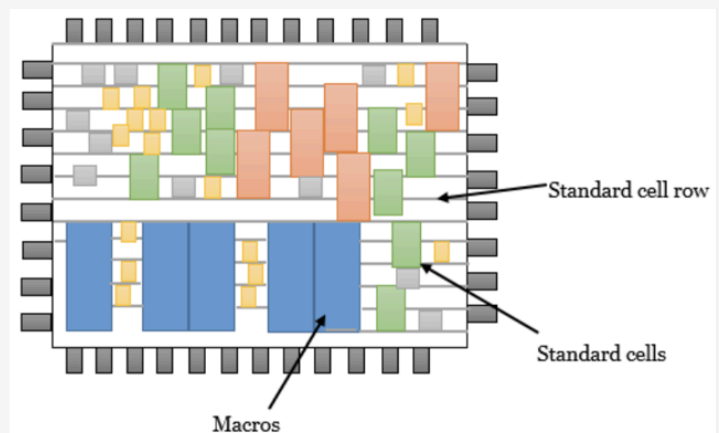
2. 背景知识

2.1. 标准单元

Standard Cell (标准单元) 是集成电路设计中常用的一种构建模块, 它指的是在数字电路中, 可以反复使用的一小块、预先设计好的电路单元。这些单元布局布线在预定义的高度, 无缝互连, 允许创建复杂的数字电路。可以把它类比为乐高积木, 每一块积木 (标准单元) 都有一个固定的形状和功能, 设计师可以通过组合不同的积木来搭建一个完整的模型 (芯片电路)。**基于标准单元库的 ASIC 设计方法**, 可以大幅缩短设计周期、提高设计可靠性, 因此已成为当前主流的工业流程。



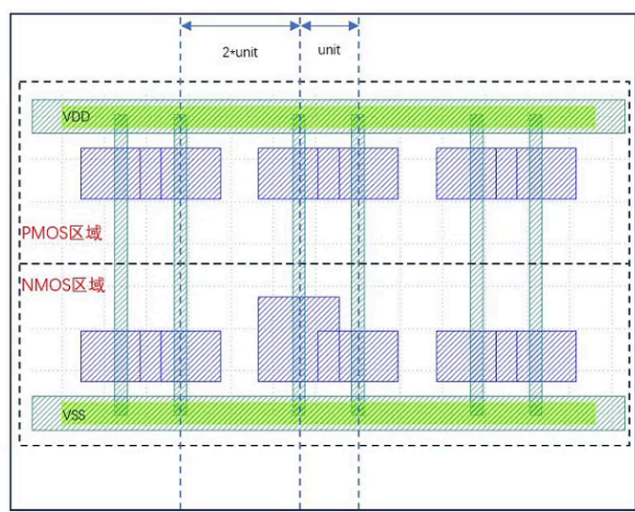
标准单元示例



基于标准单元的数字电路设计

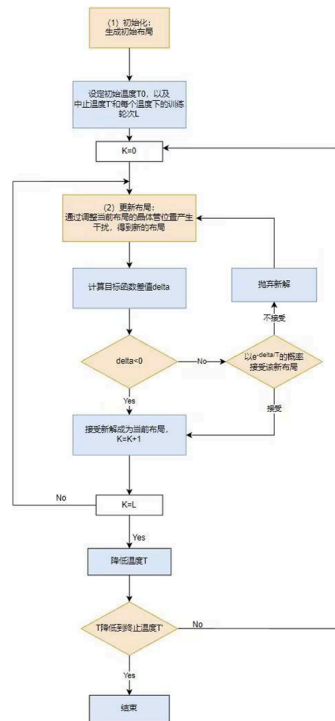
随着工艺更新、设计复杂度增加，库中单元的种类和数量迅速增长，且需要频繁针对新工艺规则进行更新。如果仍依靠版图工程师逐个单元手工绘制和维护，不仅工作量巨大、周期漫长，而且很难保证风格统一和质量稳定，因此**标准单元版图设计的自动化**变得尤为重要。

标准单元版图设计大体可以分为晶体管布局（placement）和布线（routing）两个阶段。**本实验主要聚焦前者，即在给定工艺简化模型下，如何在固定高度的单元内部对 NMOS 与 PMOS 晶体管进行合理布局：**上方一行为 PMOS，下面一行为 NMOS，中间通过栅极对齐、源漏共享等方式尽量压缩宽度，同时满足不产生有源区凹槽（notch）、不违背最小间距等工艺规则。



单元版图生成（Cell Layout Generation）正是为了解决这一问题而提出的技术路线。它以**晶体管级网表和工艺设计规则**为输入，通过算法自动给出满足设计约束的单元版图。然而，要让自动生成的单元版图在紧凑性和工艺可制造性方面接近甚至媲美经验丰富工程师的手工布局并不容易。即便在只考虑面积一个指标、且电路结构给定的情况下，晶体管级布局问题本身也是一个高度组合优化问题，存在大量**排列、翻转、共享有源区**等离散决策，搜索空间极大。本实验的模拟退火就是在上述这些排列、翻转、有源区共享以及**两行协调更新**的离散决策(详见附录)空间中，通过随机扰动 + 合法性检测 + 目标函数评分 + Metropolis 接受准则，不断搜索更优的晶体管布局。

2.2. 布局算法流程介绍

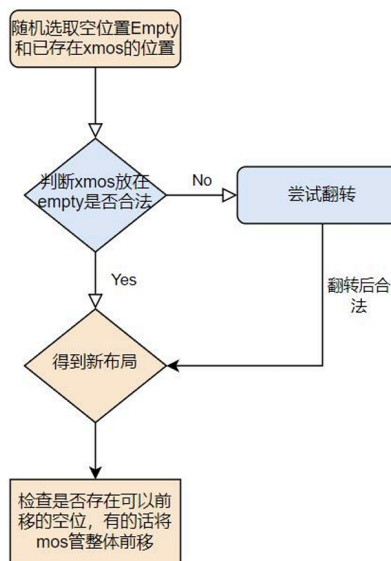


2.2.1. 初始化(eda.py)

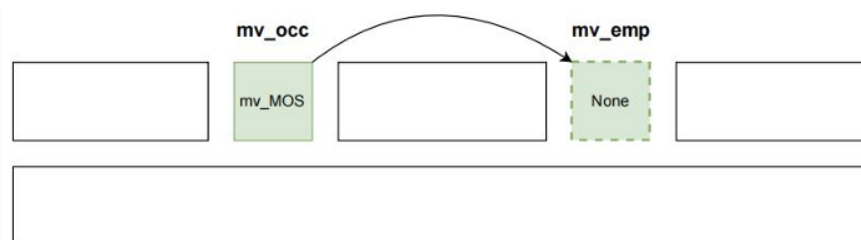
- 从文件中依次读取 NMOS 和 PMOS 的信息并分别存入到对应的 NMOS_list 或 PMOS_list 的列表中。**初始化布局不考虑过多的优化**, 直接按照网表文件中晶体管的顺序进行排布, 放置 MOS 管时, 能够产生共用的有源区则将 MOS 管相邻放置, 否则相隔一个单位长度放置。

2.2.2. 更新布局

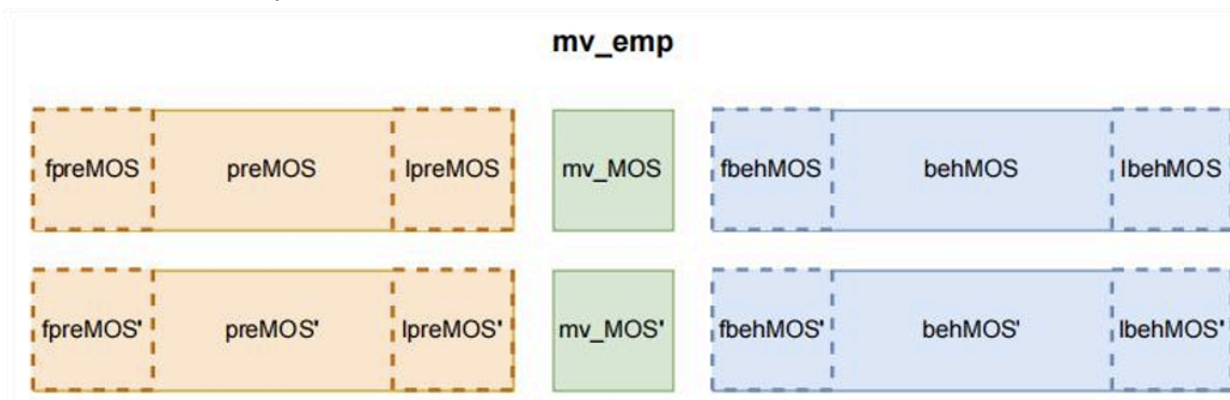
- 遍历 MOS 管的坐标列表, 分别找到**所有空位置**和**所有已被占用的位置**, 分别放到两个列表中, 然后分别从两个列表中**随机**选一个空位置 Empty 和已有晶体管 xMOS 的位置 Occupied, 检查 xMOS 放在位置 Empty 是否合法, 如果合法, 则将 xMOS 放到空位置 Occupied, 否则尝试翻转 xMOS, 或者 Empty 左右的 mos 管集合 (位置连续的一个或几个 mos 管), 来尝试找到合法布局。



2. **干扰产生**:然后随机的选择一个占用位置和一个空位置，交换占用位置和空位置，并检测交换后的布局是否合法，尝试将占用位置的MOS管移动到空位置上即为完成一次干扰。



3. **干扰合法性检测** (is_legal): 计算空位置mv_emp之前已经产生源漏共用的MOS管集合preMOS，空位置之后已经产生源漏共用的MOS管集合behMOS，以及与preMOS位置相同的另一类MOS管集合preMOS'，以及相对应的MOS管集合behMOS'.



由于MOS管可以翻转，因此可以通过翻转preMOS、mv_MOS以及behMOS尽量产生源漏共用，所以共有8种翻转情况，因此对应8种检测干扰是否合法的检测分支

本实验中的“扰动检测”并不是只判断翻转是否合法，而是以“将某个 MOS 移动到空位置”为基本扰动，在此基础上枚举 preMOS、mv_MOS、behMOS 的翻转组合，综合检查该扰动

下的排列位置是否仍然可以形成合法的源/漏共用关系，并避免 notch 等 DRC 违规。因此，它同时约束了晶体管的位置（排列）、朝向（翻转）以及有源区共享关系。

3 实验环境与文件

- 实验环境 :WSL - Ubuntu 22.04(允许自选方案完成实验)
- 语言与工具: C++17、Python 3.10.12、pybind11
- 主要文件:
 - `sa_core.cpp` : 本次实验的核心 C++ 源文件，已给出大部分实现与中文注释，请完成文件中的TODO部分。
 - `sa_bindings.cpp`: pybind11 绑定文件，`#include "sa_core.cpp"`。
 - `setup.py`: 构建脚本，将 C++ 扩展编译为 Python 模块 `sa_core`。
 - `eda.py`: 实验主程序 (Python)，负责读入网表、调用 C++ 的 `simulated_annealing_cpp`、输出 JSON 布局。
 - `cells.spi`: 某平面工艺下标准单元库中所有单元的晶体管级网表。
 - 若干 `*.json` : 运行结果（布局文件）。

4 实验内容

1. 学习并理解提供的辅助函数代码,利用提供的函数接口完成实验代码的编写
2. 在 `sa_core_stub.cpp` 中完成模拟退火主循环函数:
 - 函数签名: `simulated_annealing_cpp(...)`
 - 已提供:
 - 数据结构 `Mos`、`Pins`、`Nets`，以及布局表示 `vector<optional<Mos>>`。
 - 目标函数与子指标: `routing`，`calc_pin_access`，`calc_symmetric`，`evaluator` 等。
 - 局部合法性与几何约束: `is_legal`，`check_notch`，`update_place_ary`，`update_mos_ary` 等。
 - 邻域辅助: `compute`（返回 occupy / empty 列），`pre_MOS`，`beh_MOS` 等。
 - 你要完成标有 "// 待补充" 的代码部分
 - 调用 `is_legal` 判断本次扰动 (move) 是否合法
 - 使用 `evaluator` 计算当前解与候选解得分。
 - 实现 Metropolis 接受准则与温度更新。
 - 模拟退火的降温步骤

3. 完成函数补全后, 用 pybind11 编译 C++ 扩展,之后选择cells.spi中的标准单元进行测试生成布局文件(.json)

```
xiao@xat:/mnt/c/Users/93954/Downloads/模拟退火$ ls
AN2D2.json  cells.spi  sa_bindings.cpp  '屏幕截图 2025-11-27 154735.png'
ReadMe.txt  eda.py     sa_core.cpp      '屏幕截图 2025-11-27 154806.png'
build       evaluator.py setup.py
xiao@xat:/mnt/c/Users/93954/Downloads/模拟退火$ python3 setup.py build
running build
running build_ext
xiao@xat:/mnt/c/Users/93954/Downloads/模拟退火$ python3 eda.py cells.spi AN2D2
[main] netlist = cells.spi cell = AN2D2
[read_cell] 打开网表文件: cells.spi
[read_cell] 找到 .SUBCKT: ['.SUBCKT', 'AN2D2', 'A1', 'A2', 'Z', 'VDD', 'VSS']
[read_cell] 找到 .ENDS, 子电路读取结束
[read_cell] 子电路内容行数: 10
[main] read_cell 返回行数: 10
[init] 处理 cell: AN2D2
[pro_head] pins: ['A1', 'A2', 'Z', 'VDD', 'VSS']
[pro_row] 输入行: ['MM9', 'net14', 'A2', 'VSS', 'VSS', 'nch_mac', 'l=30.0n', 'w=0.14u']
[pro_row] 进入前 nets 列表: ['A1', 'A2', 'Z', 'VDD', 'VSS']
[pro_row] 处理后 nets 列表: ['A1', 'A2', 'Z', 'VDD', 'VSS', 'net14']
[pro_row] 输入行: ['MM8', 'net26', 'A1', 'net14', 'VSS', 'nch_mac', 'l=30.0n', 'w=0.14u']
[pro_row] 进入前 nets 列表: ['A1', 'A2', 'Z', 'VDD', 'VSS', 'net14']
[pro_row] 处理后 nets 列表: ['A1', 'A2', 'Z', 'VDD', 'VSS', 'net14', 'net26']
[pro_row] 输入行: ['MM5', 'Z', 'net26', 'VSS', 'VSS', 'nch_mac', 'l=30.0n', 'w=0.14u', '']
[pro_row] 进入前 nets 列表: ['A1', 'A2', 'Z', 'VDD', 'VSS', 'net14', 'net26']
[pro_row] 处理后 nets 列表: ['A1', 'A2', 'Z', 'VDD', 'VSS', 'net14', 'net26']
[pro_row] 输入行: ['MM5_2', 'Z', 'net26', 'VSS', 'VSS', 'nch_mac', 'l=30.0n', 'w=0.14u', '']
[pro_row] 进入前 nets 列表: ['A1', 'A2', 'Z', 'VDD', 'VSS', 'net14', 'net26']
[pro_row] 处理后 nets 列表: ['A1', 'A2', 'Z', 'VDD', 'VSS', 'net14', 'net26']
[pro_row] 输入行: ['MM6', 'net26', 'A1', 'VDD', 'VDD', 'pch_mac', 'l=30.0n', 'w=170.0n']
[pro_row] 进入前 nets 列表: ['A1', 'A2', 'Z', 'VDD', 'VSS', 'net14', 'net26']
```

4. 对生成的布局文件进行打分

```
[write_to_file] 写入文件: AN2D2.json
[write_to_file] 写入内容长度: 1494
Placement written to AN2D2.json
xiao@xat:/mnt/c/Users/93954/Downloads/模拟退火$ python3 evaluator.py AN2D2.json AN2D2 cells.spi
Cell score 94.143919 (width: 5, bbox: 4.500000, pin_access: 0.150000, symmetric: 10, drc: 10, runtime: 0s)
xiao@xat:/mnt/c/Users/93954/Downloads/模拟退火$ S
S: command not found
```

5 实验要求与报告内容

1. 代码实现要求

- 完成 `simulated_annealing_cpp` 中所有 TODO 段, 保证能够成功编译并运行。
- 可以更改eda.py中对模拟退火参数的设置
- 其他文件最好不要进行改动,若有改动,请在报告中说明

2. 实验报告内容

- 实验过程
- 结果展示
 - 展示提供的标准单元的运行结果

- 可以尝试不同参数组合（起始温度 t_0 、降温因子 decrease、每温度迭代次数 times），对结果进行对比和分析。（退火参数在eda.py中设置）
- 分析与讨论
 - SA 与简单贪心（只接受变好解）的效果差异。
 - 温度、降温速度对结果质量和运行时间的影响。
- 3. 可选做项（**不计入**额外分数）
 - 在 Python 层记录每轮温度的最优得分，画出收敛曲线（得分 vs. 温度或迭代次数）。
 - 设计并实现一种改进的温度 schedule（如线性降温、指数降温对比）。

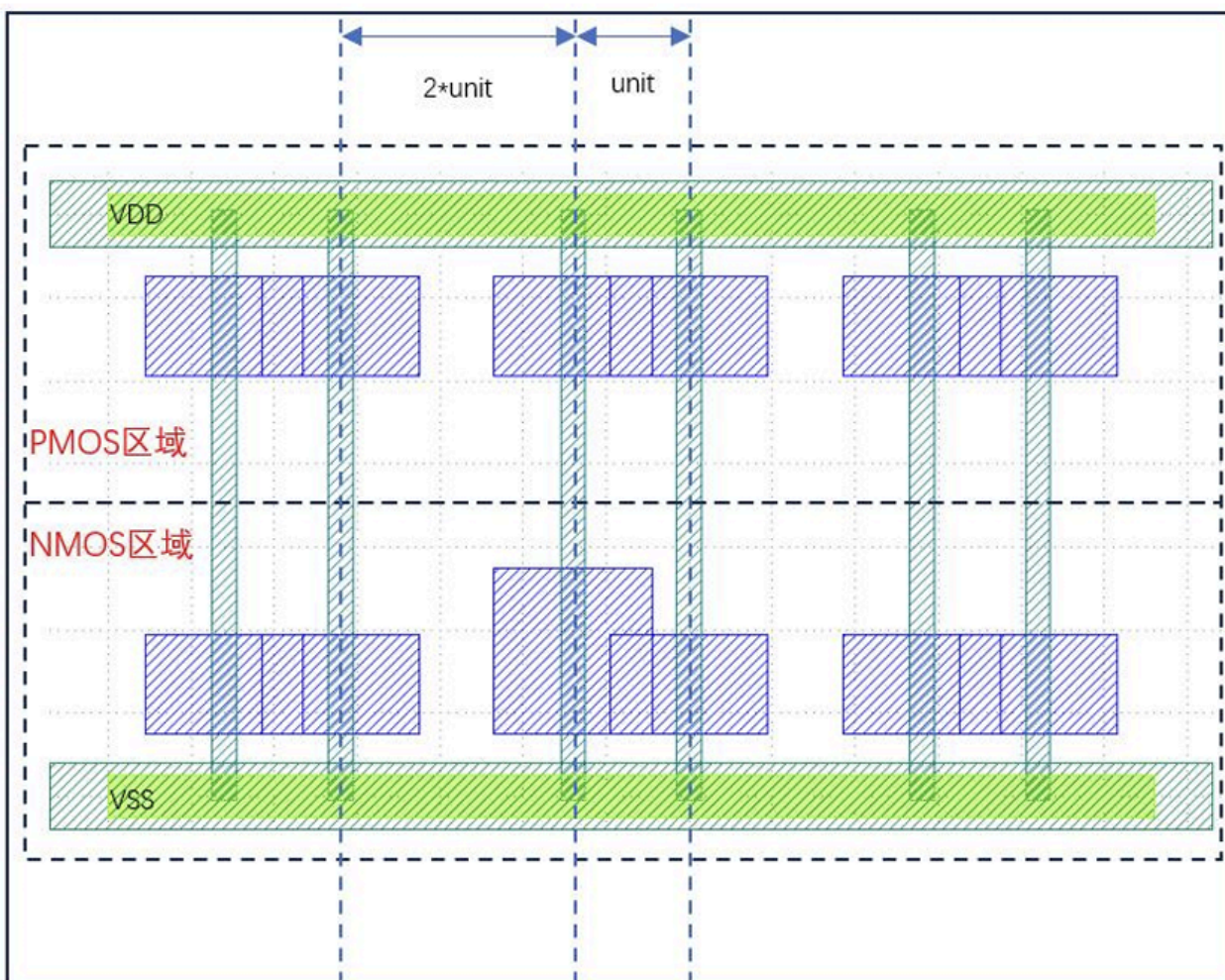
6 提交内容

- 源代码
- 实验报告
- 如有额外脚本（如画图脚本），一并提交并在报告中说明使用方法。

7 附录

7.1 标准单元布局

平面工艺下简化后的布局问题如下图所示，标准单元将被放在两条电源轨道（Power Rail）之间，高度固定，宽度不定。在标准单元的内部，晶体管按两行依次放置，PMOS晶体管在上面一行，NMOS晶体管放在下面一行，本实验所有晶体管的有源区向电源轨道对齐



注：

1. 晶体管由两个矩形表示，垂直矩形为栅极，蓝色矩形为有源区。
2. 整个标准单元被等分为两个部分，上半部分为PMOS晶体管可放置的区域，下半部分为NMOS晶体管可放置的区域。
3. 所有晶体管有源区向两侧电源轨道对齐。
4. 同一水平位置的晶体管栅极会直接相连。

7.1.1 布局时可对晶体管采用的策略

a. 排列 / 交换位置 (Placement / Move)

- 在同一行（NMOS 行或 PMOS 行）内改变 MOS 的先后顺序。
- 具体实现是：选一个“占用位置”和一个“空位置”，把某个 MOS 从占用位置搬到空位置，或者等价地视作交换两个位置的状态。
- 作用：改变哪些管子彼此相邻，从而影响能否共享有源区、是否产生 **notch**、pin 分布是否

更集中等。

b. 翻转 (Flip)

- 对单个 MOS 或一段连续的 MOS 集合做左右翻转，相当于交换其源极 S 和漏极 D 在版图中的左右位置（网表连接不变）。
- 算法在做扰动合法性检查时，会枚举 **preMOS**、**mv_MOS**、**behMOS** 是否翻转的 8 种组合。
- 作用：
 - 让“面向邻居的一侧”刚好是连接到相同 net 的端子，从而实现源/漏共用；
 - 同时避免因为某种朝向导致 notch 或 DRC 违规。

c. 决定是否共享有源区 (Diffusion Sharing)

- 对相邻的两个 MOS，如果它们相邻一侧的端子连在**同一个 net**，上下两行也能配合，就可以选择：
 - 共用扩散区：在版图上画成连续的一块有源区；
 - 不共用：中间留出最小间距，各自独立。
- 这一点在代码中是通过 **preMOS/behMOS** 集合 + **is_legal** 的条件来隐式决定的。
- 作用：
 - 共享有源区可以减少单元宽度、减少接触/扩散边界数量，但有时为了另一行的对齐、**对称性**或 DRC (notch)，会放弃共享。

7.2 输入文件解读：以与非门 (NAND2V1) 网表为例

```
.SUBCKT NAND2V1 A1 A2 ZN VDD VSS
MM1 ZN A1 net18 VSS n09_ckt W=0.12u L=30.00n
MMN1 net18 A2 VSS VSS n09_ckt W=0.12u L=30.00n
MM0 ZN A2 VDD VDD p09_ckt W=0.12u L=30.00n
MMP1 ZN A1 VDD VDD p09_ckt W=0.12u L=30.00n
.ENDS
```

1. **子电路定义行**：.SUBCKT NAND2V1 A1 A2 ZN VDD VSS
.SUBCKT：表示这是一个子电路 (Subcircuit) 的定义开始
NAND2V1：子电路的名称
A1 A2 ZN VDD VSS：子电路的外部引脚 (Port) 列表
A1/A2：与非门的两个输入引脚；
ZN：与非门的输出引脚 (“N” 表示低电平有效) ；

VDD: 电源引脚

VSS: 地引脚

2. 晶体管定义行 (共 4 行, 对应 4 个 MOS 管)

每一行描述一个 MOS 晶体管的连接关系、器件模型和尺寸参数, 格式为: **名称 D G S**

B 模型名 尺寸参数 (注: MOS 管的S/D在电路中可互换, B通常与电源 / 地相连)。

选取一行解释: MM1 ZN A1 net18 VSS n09_ckt W=0.12u L=30.00n

MM1: 晶体管的实例名称 (“M” 表示 MOS 管, “M1” 是具体编号);

ZN: 晶体管的漏极 (D), 连接到与非门输出引脚 ZN;

A1: 晶体管的栅极 (G), 连接到与非门输入引脚 A1;

net18: 晶体管的源极 (S), 连接到内部线网 (Net) net18 (非外部引脚, 是模块内部的连接节点);

VSS: 晶体管的衬底 (B), 连接到地 (NMOS 管的衬底通常接地);

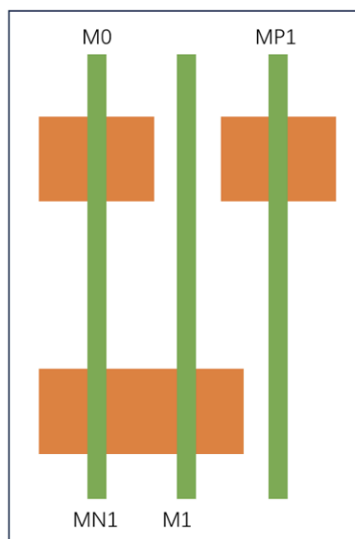
n09_ckt: 器件模型名称, “n” 表示这是NMOS 管, 09_ckt是模型的具体标识;

W=0.12u: 沟道宽度 (Width) 为 0.12 微米;

L=30.00n: 沟道长度 (Length) 为 30 纳米。

3. 子电路结束行: .ENDS

假设其布局结果如下图



那么对应晶体管信息:

- M1 = (1, 0, net18, A1, ZN, 120nm)
- MN1 = (0, 0, VSS, A2, net18, 120nm)
- M0 = (0, 1, VDD, A2, ZN, 120nm)

- $MP1 = (2, 1, ZN, A1, VDD, 120nm)$

7.3 布局算法补充介绍

7.3.1 初始化

- 从文件中依次读取 NMOS 和 PMOS 的信息并分别存入到对应的 NMOS_list 或 PMOS_list 的列表中。首先将第一个 NMOS 摆放到位置 0，接着摆放一个 PMOS，检查位置 0 对于该 PMOS 是否合法（也就是源漏漏极符合要求，且摆放不会带来源区凹槽）。摆放完成后，接着摆放下一个 NMOS……交叉着进行摆放，直到所有 MOS 管都被摆放完成。
- 初始化布局不考虑过多的优化，直接按照网表文件中晶体管的顺序进行排布，放置一 MOS 管时，能够产生共用的有源区则将 MOS 管相邻放置，否则相隔一个单位长度放置。在算法执行过程中，需要保证 PMOS 的布局数组与 NMOS 的布局数组长度相同，以检查相同位置的 PMOS 与 NMOS 的栅极。假设晶体管折叠后的 PMOS 数量为 $n1$ ，NMOS 数量为 $n2$ ，则布局宽度最差的情况为 $2 \times \max(n1, n2)$ 。

7.3.2 干扰合法性检测的算法

计算空位置 mv_emp 之前已经产生源漏共用的 MOS 管集合 $preMOS$ ，空位置 mv_emp 之后已经产生源漏共用的 MOS 管集合 $behMOS$ ，以及与 $preMOS$ 位置相同的另一类 MOS 管集合 $preMOS'$ ，与 $behMOS$ 位置相同的另一类 MOS 管集合 $behMOS'$ 。



由于 MOS 管可以翻转，因此可以通过翻转 $preMOS$ 、 mv_MOS 以及 $behMOS$ 尽量产生源漏共用，因此这三者是否翻转的情况有：

preMOS	mv_MOS	behMOS	满足条件	解释
0	0	0	<pre>((!preMOS=None) (!preMOS.d=mv_MOS.s)) && ((!behMOS=None) (!behMOS.s=mv_MOS.s)) && ((mv_MOS' !=None) (mv_MOS.g=mv_MOS'.g))</pre>	1. mv_MOS 与 preMOS 源漏共用 2. mv_MOS 与 behMOS 源漏共用 3. mv_MOS 与 mv_MOS' 栅极共用
0	0	1	<pre>((!preMOS=None) (!preMOS.d=mv_MOS.s)) && ((!behMOS=None) (!behMOS.d=mv_MOS.s)) && ((mv_MOS' !=None) (mv_MOS.g=mv_MOS'.g))</pre>	1. mv_MOS 与 preMOS 源漏共用 2. mv_MOS 与 翻转的 behMOS 源漏共用 3. mv_MOS 与 mv_MOS' 栅极共用
0	1	0	<pre>((!preMOS=None) (!preMOS.d=mv_MOS.d)) && ((!behMOS=None) (!behMOS.s=mv_MOS.s)) && ((mv_MOS' !=None) (mv_MOS.g=mv_MOS'.g))</pre>	1. 翻转的 mv_MOS 与 preMOS 源漏共用 2. 翻转的 mv_MOS 与 behMOS 源漏共用 3. mv_MOS 与 mv_MOS' 栅极共用
0	1	1	<pre>((!preMOS=None) (!preMOS.d=mv_MOS.d)) && ((!behMOS=None) (!behMOS.d=mv_MOS.s)) && ((mv_MOS' !=None) (mv_MOS.g=mv_MOS'.g))</pre>	1. 翻转的 mv_MOS 与 preMOS 源漏共用 2. 翻转的 mv_MOS 与 翻转的 behMOS 源漏共用 3. mv_MOS 与 mv_MOS' 栅极共用
1	0	0	<pre>((!preMOS=None) (!preMOS.s=mv_MOS.s)) && ((!behMOS=None) (!behMOS.s=mv_MOS.d)) && ((mv_MOS' !=None) (mv_MOS.g=mv_MOS'.g))</pre>	1. mv_MOS 与 翻转的 preMOS 源漏共用 2. mv_MOS 与 behMOS 源漏共用 3. mv_MOS 与 mv_MOS' 栅极共用
1	0	1	<pre>((!preMOS=None) (!preMOS.s=mv_MOS.s)) && ((!behMOS=None) (!behMOS.d=mv_MOS.d)) && ((mv_MOS' !=None) (mv_MOS.g=mv_MOS'.g))</pre>	1. mv_MOS 与 翻转的 preMOS 源漏共用 2. mv_MOS 与 翻转的 behMOS 源漏共用 3. mv_MOS 与 mv_MOS' 栅极共用
1	1	0	<pre>((!preMOS=None) (!preMOS.d=mv_MOS.d)) && ((!behMOS=None) (!behMOS.s=mv_MOS.s)) && ((mv_MOS' !=None) (mv_MOS.g=mv_MOS'.g))</pre>	1. 翻转的 mv_MOS 与 翻转的 preMOS 源漏共用 2. 翻转的 mv_MOS 与 behMOS 源漏共用 3. mv_MOS 与 mv_MOS' 栅极共用
1	1	1	<pre>((!preMOS=None) (!preMOS.s=mv_MOS.s)) && ((!behMOS=None) (!behMOS.d=mv_MOS.d)) && ((mv_MOS' !=None) (mv_MOS.g=mv_MOS'.g))</pre>	1. 翻转的 mv_MOS 与 翻转的 preMOS 源漏共用 2. 翻转的 mv_MOS 与 翻转的 behMOS 源漏共用 3. mv_MOS 与 mv_MOS' 栅极共用

注:

- MOS.s表示 MOS 管源极，MOS.g表示 MOS 管栅极，MOS.d表示 MOS 管漏极
- 对于翻转组合的情况，0 表示不进行翻转，1 表示进行翻转

- 需要翻转同位置的 MOS' 集合：
 - 若 MOS 集合内的 MOS 管数量为 0 或 1 个，则无需翻转；
 - 否则翻转相同位置的 MOS" 集合，并判断能否与mv_MOS'产生源漏共用。
- 若能够通过某一翻转情况的条件，则可以将mv_MOS放置在空位置。
- 在翻转 MOS 集合时，需要注意的是：需要交换每个内部 MOS 的源极和漏极；在 MOS 集合的 MOS 管数量大于等于 2 时，也需要反转 MOS 集合

7.4 输出的布局json文件

- 以每个MOS晶体管的名字为key（若为折叠后的晶体管，加上_finger\${id}后缀名用以区分，具体格式参考example.json），value为字典，各属性含义如下表：

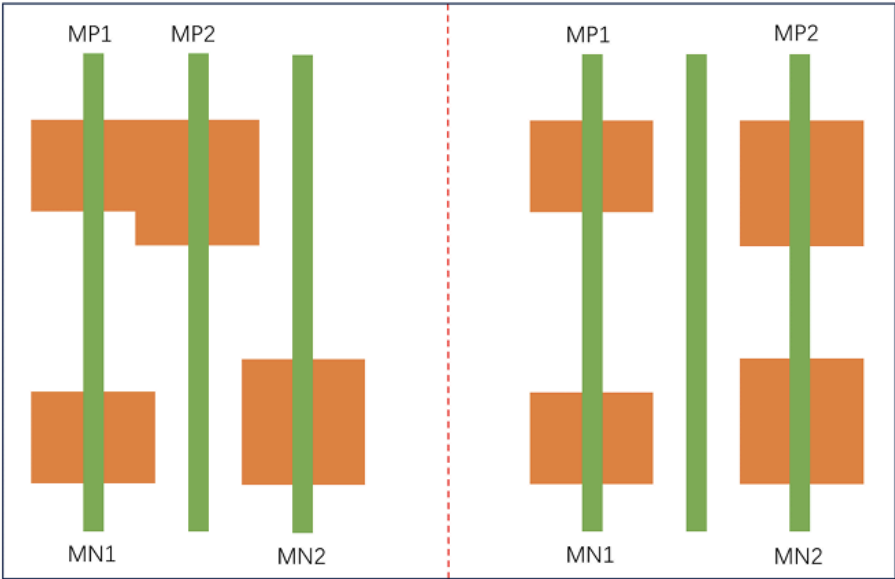
属性字段	含义	值范围
x	水平方向的相对位置，值为整数。 放在最左侧的 MOS 晶体管 x=0	≥ 0 ，整数
y	0 表示 NMOS，1 表示 PMOS	0 或 1
source	MOS 晶体管左侧所连接线网	字符串
gate	MOS 晶体管栅极所连接线网	字符串
drain	MOS 晶体管右侧所连接线网	字符串
width	MOS 晶体管的沟道宽度	≥ 0 ，整数，单位为 nm

7.5 评价布局好坏的参数(辅助理解提供的c++中的各个辅助函数)

评价指标	备注
布局宽度	标准单元总的宽度
布线复杂度	所有线网所占宽度之和，不考虑VDD，VSS
对称性	栅极不配对的晶体管数量，越小表示对称性越好
pin 密度	pin 位置的方差。当存在多个位置时，选择距离周围pin 最远的 x 坐标。这里也不考虑 VDD，VSS。
DRC	有源区凹槽。若存在该情况，则为 0 分，否则满分

a. 对称性

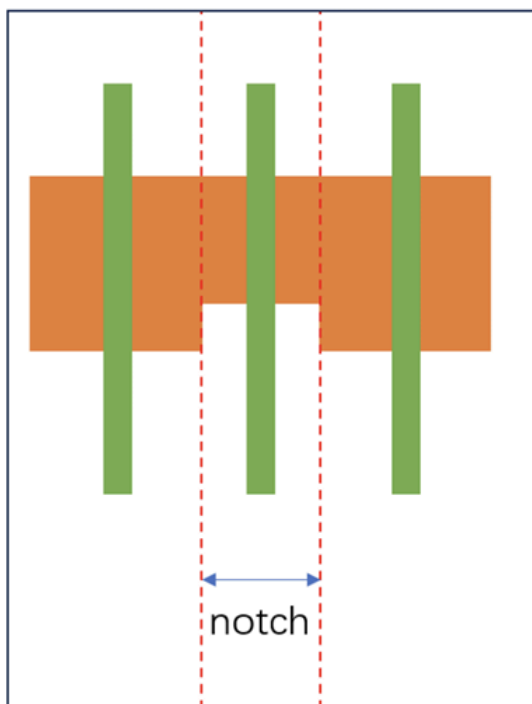
- 左图中MP2未与MN2栅极配对，虽然宽度和右图相等，但是右图的对称性更好。本实验中对称性得分就是计算栅极未配对的晶体管数量。例如左图对称性得分 $10-2=8$ ，右图对称性得分 $10-0=10$ 。



b. DRC (notch检查)

- 当左右两侧的晶体管宽度大于中间晶体管时，将产生有源区凹槽，导致min spacing设计规则违背。每个标准单元的得分将根据上述六个评价指标加权后计算得出。每个标准单元在最终得分中的占比将由其MOS晶体管的数量决定，计算所有标准单元的加权分数之和，得

出最终分数。



8 参考文献

- [1] 马琪,王旭;CMOS 单元版图生成中的晶体管布局算法[J];电路与系统学报;2004 年04 期.
- [2] 马琪,罗小华,严晓浪;CMOS 单元版图生成算法综述[J];微电子学;2001 年 03 期.
- [3] Schneider, Jan: Transistor-Level Layout of Integrated Circuits. - Bonn, 2014.-Dissertation, Rheinische Friedrich-Wilhelms-Universität Bonn.
- [4] Kahng, A.B. et al. (2022) . Springer